Implementação 5

Sincronização do Metrô

bgs@cesar.school · 4 de novembro de 2026

1. Objetivo

Coordenar o embarque de passageiros nas estações da METROREC. Cada passageiro deve obter um assento antes de embarcar, e o vagão deve aguardar a última confirmação antes de partir. A entrega reúne o código em C, os testes executáveis e os registros da validação em Linux.

2. Checklist de requisitos

Requisito do enunciado	Status	Como testar ou conferir
Inicialização da estação	OK	station_init inicializa o mutex, as duas condições e os três contadores.
Reserva de um assento por passageiro	OK	Runner, E3. Os embarques conferem com as vagas e com os passageiros restantes.
Partida após as confirmações	OK	Teste complementar, E4. A mantém uma reserva pendente enquanto B conclui sua viagem.
Embarques simultâneos	OK	Em E4, o segundo passageiro de A obtém vaga antes de o primeiro confirmar.
C em ambiente POSIX	OK	E1 identifica o Ubuntu e o GCC. E2 compila com a opção -pthread.
Sete funções pthread exigidas	OK	sync_car.c usa mutex_init/lock/unlock e cond_init/wait/signal/broadcast, com o prefixo pthread_.
Um mutex por estação	OK	sync_car.h declara um único pthread_mutex_t na struct station.
Somente mutex e condições	OK	Em sync_car.c, as operações de sincronização se limitam ao mutex e às duas condições POSIX.
Bloqueio sem espera ocupada	OK	Tanto o vagão quanto o passageiro dormem em pthread_cond_wait ao aguardar uma mudança.
Um vagão por vez em cada estação	OK	E4 mantém um vagão em A e outro em B. Cada estação recebe um vagão por vez.
Makefile, executável e limpeza	OK	E2 gera sync_car, E6 executa os testes e E7 remove objetos e executáveis.

Os marcadores E1 a E7 correspondem às etapas datadas de evidencias.log.

3. Como reproduzir

No Ubuntu, abra um terminal no diretório que contém bgs.tar. A sequência abaixo extrai a entrega, compila do zero, executa o runner, roda os dois testes e limpa os binários.

```
tar -xf bgs.tar
cd bgs
make clean
make
./sync_car
make test
make clean
```

O runner imprime “Looks good!” quando contabiliza os 100 passageiros. O segundo teste imprime duas linhas iniciadas por “OK”. A compilação padrão emite um aviso no runner fornecido, detalhado em E1 e E2.

4. Arquitetura

Arquivo	Responsabilidade
sync_car.h	Expõe struct station e os quatro protótipos aos dois executáveis.
sync_car.c	Inicializa a estação e coordena chegada, reserva e confirmação.
sync_car-runner.c	Runner da atividade. Cria 100 passageiros, sorteia vagas e confere a lotação de cada viagem.
test_stations.c	Testa capacidade negativa, chegada durante o embarque, duas reservas pendentes e estações independentes.
Makefile	Compila sync_car, oferece o alvo test e remove os arquivos gerados com clean.
README.md / bgs.pdf	Reúnem as instruções de execução e o relatório, respectivamente.
evidencias.log / evidencias/	Guardam a sessão de validação e as duas capturas do painel.

Decisões técnicas

1. Reserva atômica. Sob o único mutex, station_wait_for_car retira um passageiro da fila (waiting), consome uma vaga (seats) e registra uma confirmação pendente (boarding). O passageiro solta o mutex antes do embarque físico, permitindo que outro reserve.

2. Dois motivos de espera. O vagão anuncia vagas com broadcast. Cada passageiro acordado testa seats novamente sob o mutex. Se outro já ocupou a última vaga, ele volta a dormir. A reavaliação em while também cobre despertares espúrios.

3. Partida. station_on_board reduz boarding e sinaliza o vagão quando o contador zera. O vagão reavalia a fila e as vagas: só parte quando todas as reservas estão confirmadas e a fila acabou ou as vagas acabaram. Antes de soltar o mutex, zera as vagas que sobraram.

5. Estratégias e diário de desenvolvimento

5.1 Estratégia da semana

ID	Estratégia	Contexto	Motivo da troca
S1	Um mutex e duas condições	Separar fila, vagas livres e embarques pendentes para decidir a partida.	A mesma estratégia foi usada até a validação final.

5.2 Diário de tentativas

A tabela cobre a sessão gravada em 04/11/2026. As horas são de Fortaleza (UTC−3). Cada marcador pode ser localizado diretamente em evidencias.log.

#	Estr.	Tentativa	Resultado	Hipótese / causa	Quando	Evidência
1	S1	Compilar com -Werror	Falha com código 2.	-Werror promove a erro o aviso sobre foo, parâmetro sem uso no runner.	20:01:07	E1, linhas 22–32
2	S1	Limpar e usar o Makefile padrão	sync_car gerado.	—	20:01:08	E2, linhas 35–45
3	S1	Executar o runner original	100 embarques confirmados.	—	20:01:10	E3, linhas 48–58
4	S1	Capacidade -1 e estações A/B	Duas verificações OK.	—	20:01:10	E4, linhas 61–67
5	S1	Repetir os dois programas	40/40 no runner e 100/100 em A/B.	—	20:01:11	E5, linhas 70, 193 e 396
6	S1	Rodar make test	Os dois executáveis passaram.	—	20:01:24	E6, linhas 398–416
7	S1	Limpar e calcular hashes	Binários removidos e fontes identificados.	—	20:01:25	E7, linhas 419–434

Dificuldade de sincronização

O caso decisivo é o intervalo entre a reserva e a confirmação. Em A, há três vagas. O primeiro passageiro reserva e o segundo chega antes de qualquer confirmação. Um confirma e o outro permanece em embarque enquanto B completa sua própria viagem. Só depois da última confirmação A pode partir com uma vaga livre.

Escopo verificado

As quatro funções previstas foram implementadas. O runner sorteia as capacidades e o escalonador decide a ordem das threads. As 140 execuções do lote E5 passaram. O lote conta execuções dos programas, e uma mesma ordem de escalonamento pode se repetir.

6. Evidências

evidencias.log foi gravado com script -a em um pseudoterminal do Ubuntu. A sessão começa com date, whoami e pwd e termina com código 0. As imagens reproduzem o texto do painel tmux capturado durante a execução.

Figura 1. Compilação estrita interrompida no runner fornecido

```
$ grep PRETTY_NAME /etc/os-release
PRETTY_NAME="Ubuntu 26.04 LTS"
$ cc -dumpfullversion
15.2.0
$ make --version
$ head -1
GNU Make 4.4.1
$ make clean
rm -f sync_car test_stations sync_car.o sync_car-runner.o test_stations.o
$ make 'CFLAGS=-O2 -Wall -Wextra -Werror -std=gnu11'
cc  -O2 -Wall -Wextra -Werror -std=gnu11 -pthread -c sync_car.c -o sync_car.o
cc  -O2 -Wall -Wextra -Werror -std=gnu11 -pthread -c sync_car-runner.c -o sync_car-runner.o
sync_car-runner.c: In function ‘alarm_handler’:
sync_car-runner.c:54:24: error: unused parameter ‘foo’ [-Werror=unused-parameter]
   54 | void alarm_handler(int foo)
      |                    ~~~~^~~
cc1: all warnings being treated as errors
make: *** [Makefile:16: sync_car-runner.o] Error 1
$ printf 'Código de saída do make estrito: %s\n' 2
Código de saída do make estrito: 2
$ set +x
```

E1, linhas 22–32. O GCC aponta foo em alarm_handler, na linha 54 do runner. Com as opções padrão, o mesmo diagnóstico aparece como aviso e o executável é gerado (E2, linha 45).

Figura 2. Validação final pelo alvo make test

```
[E6] 2026-11-04 20:01:24 -0300 | Validação final dos dois testes
$ make test
./sync_car
car entering station with 4 free seats
car departed station with 4 new passenger(s) (expected 4)
car entering station with 4 free seats
car departed station with 4 new passenger(s) (expected 4)
car entering station with 9 free seats
car departed station with 9 new passenger(s) (expected 9)
car entering station with 1 free seats
car departed station with 1 new passenger(s) (expected 1)
car entering station with 47 free seats
car departed station with 47 new passenger(s) (expected 47)
car entering station with 48 free seats
car departed station with 35 new passenger(s) (expected 35)
Looks good!
./test_stations
OK: capacidade -1 retorna sem conceder uma vaga.
OK: duas reservas simultâneas e duas estações independentes.
$ set +x
```

E6, linhas 398–416. O último vagão ofereceu 48 vagas para 35 passageiros restantes. Em seguida, o segundo executável confirmou o caso inválido e a independência das estações.

7. Uso de IA

O Codex analisou o enunciado e o guia, escreveu a implementação, o Makefile e o teste complementar, executou os testes em Ubuntu e organizou as evidências e o relatório. O runner foi fornecido com a atividade.

Pedido principal enviado à IA

“analise essa documentaçao veja tudo que precisa pergunte o que precisa perguntar paara mim e dps faça ela toda”

Validação pessoal: o estudante informou que ainda não executou ou revisou os testes por conta própria. As verificações descritas nas seções 5 e 6 foram executadas pelo Codex.

8. Reflexão final

A última vaga pode estar reservada enquanto o passageiro ainda procura o assento. Por isso, a partida depende das confirmações pendentes, além da fila e das vagas livres. O teste com A e B tornou essa diferença observável: B partiu enquanto A ainda aguardava. Em uma próxima implementação, convém testar essa ordem desde o início. Também vale separar cedo os avisos do runner dos avisos da solução, como mostrou a tentativa com -Werror.

9. Checklist final de entrega

Item	Confirmação
Compilação do zero com os comandos do relatório	Sim · E2
Testes executados e evidencias.log gravado	Sim · E3–E7
Capturas de erro e validação final incluídas	Sim · figuras 1 e 2
Caso limite (zero vagas) e caso inválido (−1)	Sim · runner e E4
Seção de uso de IA preenchida	Sim · seção 7
Relatório revisado com resultados e evidências específicos	Sim · seções 2, 5 e 6

10. Se eu tivesse mais 2 horas

A primeira hora seria dedicada a um teste de disputa pela última vaga. Ele colocaria vinte passageiros em uma estação e enviaria vinte vagões com um assento cada. O critério seria uma confirmação por viagem e vinte no total. Registrar o identificador de cada passageiro permitiria conferir também se todos viajaram uma única vez. Na segunda hora, o teste complementar seria recompilado com ThreadSanitizer. A meta seria repetir o cenário A/B cem vezes e investigar cada acesso compartilhado apontado pelo detector.